오픈소스소프트웨어
중간고사 요약본

챕터별 정리

01 오픈소스소프트웨어 1주차 핵심 요약

02 오픈소스소프트웨어 2주차 핵심 요약

03 3주차 핵심 요약: AWS EC2 & 인프라 구조

04 4주차 핵심 요약: 리눅스 시스템 관리 & 패키지

05 5주차 핵심 요약: 리눅스 핵심 명령어 & 시스템 활용

06 [Week 6] Git & GitHub - 기초 요약 노트

07 [Week 7] Git & GitHub - 원격 저장소 전체 요약 노트

CHAPTER 01

오픈소스소프트웨어 1주차 핵심 요약

1. AI 시대와 오픈소스의 필요성

- 단순 코딩은 AI 도구(Copilot, Cursor 등)가 대체 -> 기업은 창의성, 문제해결력, 협업 역량을 요구
- 주요 AI 인프라 및 도구는 전부 오픈소스 기반:
  - `PyTorch / TensorFlow`: 딥러닝/머신러닝 프레임워크
  - `Linux`: AI 서버 표준 OS
  - `Kubernetes`: 컨테이너 배포 및 운영 표준
  - `Hugging Face`: AI 모델/데이터셋 공유 플랫폼
- 개발자의 공용어: Git, Linux, Docker
- 오픈 웨이트(Open-weight) 모델: AI 학습 결과물인 가중치(weight) 파일 자체를 공개해 누구나 실행·수정·재학습 가능하게 한 모델 (예: Meta Llama, DeepSeek, Qwen 등)
- GitHub 기록(커밋, PR, 코드 리뷰 등)이 말뿐인 이력서보다 검증 가능한 강력한 포트폴리오가 됨

2. 용어 및 개념 비교

- Free Software (자유 SW): FSF(리처드 스톨만) 중심 / 윤리적 관점 / 4대 자유 보장 / GPL 중심
- Open Source Software (OSS): OSI(에릭 레이먼드) 중심 / 실용적 관점 / 소스 공개를 통한 검증과 품질 향상
- Proprietary / Closed Source: 소스코드 비공개 SW
- 주의: 카카오톡, 알약처럼 무료 프로그램이어도 소스가 비공개면 OSS가 아님!

3. 대표 라이선스 3종

- MIT: 가장 느슨함 (출처/저작권 표시만 하면 자유롭게 사용·수정·재배포 가능)
- Apache 2.0: MIT 특징 + 특허(Patent) 조항 포함
- GPL: 카피레프트(Copyleft) / 파생물(2차 저작물)도 반드시 동일한 GPL로 소스코드 공개 의무

4. OSS 핵심 원칙 5가지

- 자유 재배포 / 소스코드 공개 / 파생 저작물 허용 / 무차별 원칙 / 라이선스 독립성

5. 주요 역사 및 연도 (시험 빈출)

- 1983: 리처드 스톨만, GNU 프로젝트 시작 (Free Software 개념)
- 1985: FSF (자유소프트웨어재단) 설립
- 1989: 최초의 카피레프트 라이선스 GNU GPL v1 발표
- 1991: 리누스 토발즈, Linux 커널 공개
- 1998: 에릭 레이먼드, OSI 설립 ("Open Source" 용어 확산)
- 2005: 리누스 토발즈, Git 최초 공개
- 2008: GitHub 창립 (Pull Request 기반 협업 확산)

6. 생태계 참여 및 ALM 흐름

- 참여자 성장: Contributor (기여자) -> Committer (코드 리뷰 및 Merge/병합 권한을 가진 핵심 관리자)
- 오픈소스 실무 흐름 (ALM 생명주기):
  1) 이슈 등록 -> 2) 로컬 개발 -> 3) PR 제출 -> 4) 코드 리뷰 & CI -> 5) 품질 검사 -> 6) 병합·배포

CHAPTER 02

오픈소스소프트웨어 2주차 핵심 요약

<OSS 2장: 라이선스 (License)>

1. 권리의 구분

- 저작권 (Copyright): 창작 시 자동 발생. 저작자 허락 필요
- 특허권 (Patent): 출원 후 인정받는 독점 권리
- 라이선스 (License): 저작권자가 사용자에게 내리는 '사용/수정/배포 허가'

2. 최근 트렌드 & AI 리스크

- Permissive 라이선스(MIT, Apache 등) 비중이 약 78%로 압도적 (GPL 비중은 감소)
- AI 코딩 도구(Copilot, Cursor)의 위험: 학습 데이터의 카피레프트 코드가 출처 표시 없이 생성될 수 있어 라이선스 세탁 (License Laundering) 및 충돌 리스크 발생

3. 공통 준수사항 3가지

- 저작권 문구 유지 / 상표권(Trademark) 보호 / 라이선스 양립성(Compatibility) 확인

4. 라이선스 스펙트럼 (★시험 필수)

- Permissive (자유로움): MIT, BSD, Apache 2.0
  - 소스코드 공개 의무 없음. 상용 SW 활용에 가장 유리
  - Apache 2.0: Patent Grant (특허 무상 허용) 및 Patent Retaliation (특허 소송 시 라이선스 종료) 포함
- Weak Copyleft (수정한 부분만 공개): LGPL, MPL
  - LGPL: 동적 링크(Dynamic Linking) 시 응용프로그램 소스 공개 의무 없음 (정적 링크는 오브젝트 제공 필요)
  - MPL: 수정한 파일(파일 단위)만 공개 의무,다른 라이선스와 결합가능
- Strong Copyleft (전체 공개 의무): GPL, AGPL
  - GPL v2.0: 라이브러리 링크/수정 시 전체 소스 공개 및 GPL 배포 의무 (Linux 커널 적용)
    (내부적으로만 사용하는 경우에는 공개의무없음)
  - GPL v3.0: Tivoization(기기 내 수정본 설치/실행 막는 것) 금지, Apache 2.0과 양립 가능,특허조항추가

5. 라이선스 양립성 & 이중 라이선스

- 양립성: 서로 다른 라이선스 코드를 결합할 때 의무 충돌 여부 (GPLv2-Apache2.0 양립 불가, GPLv3-Apache2.0 양립 가능)
- 이중 라이선스 (Dual License): 하나의 코드에 두 라이선스 적용 (예: MySQL - GPL 2.0 or 상용 라이선스)

<OSS 3-1장: 클라우드 컴퓨팅 & AWS 계정>

1. 클라우드 컴퓨팅 정의 & 특징

- IT 자원(컴퓨팅, DB, 스토리지 등)을 인터넷을 통해 빌려 쓰고 사용한 만큼 지불(Pay-as-you-go)하는 방식
- 4대 이점: 비용 효율성, 확장성(Scalability), 신뢰성/가용성(Fault Resilient), 빠른 글로벌 배포

2. On-Premise vs Cloud

- On-Premise: 회사가 직접 서버 인프라 구축 및 유지보수 (초기 비용 큼, 고정 비용)
- Cloud: 필요할 때 연결하여 사용, 종량제 요금, 유지보수는 클라우드 사업자 담당

3. 클라우드 시장 점유율

- AWS(1위) -> Microsoft Azure(2위) -> Google Cloud(3위)

4. AWS 계정 보안 모범 사례 (★보안 수칙)

- 루트(Root) 사용자: 결제, 계정 삭제 등 모든 권한 보유 -> 일상적 사용 절대 금지!
- 필수 보안 조치:
  1) 루트 계정에 MFA(Multi-Factor Authentication, 다단계 인증) 반드시 설정
  2) 평소 실습/작업용 IAM(Identity and Access Management) 관리자 사용자 생성 후 IAM으로 로그인

CHAPTER 03

3주차 핵심 요약: AWS EC2 & 인프라 구조

<1. 글로벌 인프라 구조 (Region & AZ)>

1. Region (리전)

- 전 세계에 분산된 독립적인 데이터센터 클러스터 (지리적 위치 단위)
- 예: 서울 리전 (ap-northeast-2)
- 리전 선택 시 기준: 지연 시간(Latency), 법적 규제, 제공되는 서비스 등

2. Availability Zone (AZ, 가용 영역)

- 하나의 Region 안에 물리적으로 완벽히 분리되어 있는 데이터센터
- 서울 리전에는 현재 4개의 AZ가 존재 (ap-northeast-2a, 2b, 2c, 2d)
- AZ 배치 전략:
  - 가용성 최대화: N개의 서버를 N개의 다른 AZ에 각각 분산 배치 (AZ 하나가 마비되어도 서비스 유지)
  - 가용성 + 비용 균형: 2개 이상의 AZ에 나눠 배치 (실무에서 가장 흔한 절충안)

<2. 비용 관리 (Cost Management)>

1. AWS Budgets

- 사용자가 설정한 예산 임계값을 초과할 때 알림을 받도록 설정하는 서비스
- 실습 계정 추천: '제로 지출 예산 ($0.01 초과 시 즉시 알림)'을 설정하여 실수로 비용이 청구되는 것 방지

<3. EC2 인증 원리 (Key Pair & SSH)>

1. 공개키 암호화 방식 (Public-Key Cryptography)

- Key Pair 구성: Public Key (공개키) + Private Key (개인키, .pem 파일)
- 서버(EC2 인스턴스 ~/.ssh 폴더)에는 'Public Key'가 저장됨
- 사용자 컴퓨터(SSH Client)는 'Private Key (.pem)'를 가지고 접속
- 인증 방식: 사용자 개인키로 난수(Challenge)를 암호화해 보내면, EC2가 공개키로 복호화하여 본인 확인
- 권한 제한: Private Key 파일은 보안상 권한을 제한해야 접속 가능 (Linux/Mac: chmod 400 key.pem

<4. 네트워크 & 보안 구조 (VPC & Security Group)>

1. 네트워크 계층 구조 (★시험 단골)

- Region -> VPC -> Subnet -> Instance
- VPC (Virtual Private Cloud): AWS 내 나만의 독립된 가상 네트워크 공간
- Subnet (서브넷): VPC 내부를 분할한 네트워크 구역
  - 하나의 Subnet은 물리적으로 단 '1개의 AZ(가용 영역)'에만 속함
- Internet Gateway: Subnet을 외부 인터넷과 연결해 주는 관문

2. Security Group (보안 그룹)

- EC2 인스턴스 바로 앞단에서 트래픽을 제어하는 '가상 방화벽'
- 인바운드 규칙 (들어오는 트래픽) 지정:
  - SSH 접속: Port 22
  - HTTP 웹 접속: Port 80
  - HTTPS 보안 웹: Port 443
- 소스 '0.0.0.0/0': 전 세계 모든 IP 주소에서의 접근을 허용

<5. EC2 생성 및 인스턴스 관리>

1. 주요 개념

- AMI (Amazon Machine Image): OS 설치 이미지 + 기본 세팅값 (예: Amazon Linux 2023, Ubuntu 등)
- Instance Type (인스턴스 유형): CPU, RAM 등 하드웨어 스펙 규격 (예: t3.micro -> vCPU 2, RAM 1GiB)
- EBS (Elastic Block Store): EC2에 붙이는 SSD/HDD 형태의 루트 스토리지 볼륨 (예: gp3)

2. 인스턴스 상태 변경 및 비용 차이 (★주의사항)

- 중지 (Stop): CPU/메모리 컴퓨팅 요금은 중단되지만, 저장장치(EBS) 비용은 계속 청구됨 (나중에 재시작 가능)
- 종료 (Terminate): 인스턴스 및 붙어있는 EBS 볼륨까지 완전 삭제되어 비용이 완전 중단됨 (복구 불가)

CHAPTER 04

4주차 핵심 요약: 리눅스 시스템 관리 & 패키지

<1. 리눅스 개요 & 계층 구조 (Linux Basics)>

1. 리눅스의 정의 & 라이선스

- Linus Torvalds가 처음 개발한 Free Unix 계열 오픈소스 운영체제
- 리눅스 커널 라이선스: GPLv2 (소스코드가 자유롭게 공개됨)

2. 리눅스 계층 구조 (★시험 단골)

- Application (응용 프로그램): GNOME/KDE, Apache, Mail Server 등
- Shell (명령 해석기): 사용자 명령을 해석해 커널에 전달 (예: bash, csh 등)
- Kernel (핵심): 메모리/프로세스/하드웨어(CPU, RAM, HDD) 관리

3. 대표적인 배포판 계열 및 특징

- Red Hat 계열: RHEL(공식 상용), CentOS Stream(개발판), Rocky Linux / AlmaLinux(CentOS의 대체 무료 배포판), Fedora(오픈소스 프로젝트)
  - 참고: 전통적인 CentOS Linux는 2024년 6월 종료(EOL)되어 Rocky/Alma가 이를 승계함.
- Debian 계열: Debian, Ubuntu (데스크톱 친화적, 풍부한 커뮤니티)
- 패키지 관리자 비교:
  - Red Hat 계열: .rpm 포맷 / rpm, yum, dnf(최신) 관리 도구
  - Debian 계열: .deb 포맷 / dpkg, apt(최신), apt-get 관리 도구

<2. vi 에디터 (vi Editor)>

1. vi 에디터의 3가지 모드 (★시험 단골)

- Command Mode (명령 모드): vi 실행 시 기본 모드, 커서 이동 및 텍스트 삭제/복사
  - 커서 이동: h(좌), j(하), k(상), l(우)
  - 편집 명령: x(한 글자 삭제), dd(한 줄 삭제), yy(한 줄 복사), p(붙여넣기)
- Insert Mode (입력 모드): i, a, o 키로 진입, Esc 누르면 명령 모드로 복귀
- Ex Mode (마지막 줄 모드): ':' 키로 진입, 저장/종료 및 검색/치환
  - :w (저장), :q (종료), :wq (저장 후 종료), :q! (강제 종료)
  - /text (text 아래 방향 검색), ?text (text 위 방향 검색)
  - :s/old/new/g (현재 줄 전체 치환), :%s/old/new/g (파일 전체 치환)

<3. 패키지 관리자 (RPM & DNF)>

1. RPM (Red Hat Package Manager)

- 저수준(primitive) 패키지 관리 도구 (.rpm 확장자)
- 주요 옵션: -i (설치), -U (업그레이드), -e (삭제), -q (조회), -v (상세 출력), -h (진행률 #)
  - 예: rpm -Uvh package.rpm (상세 출력 및 진행률을 보며 업그레이드/설치)
- 단점: 패키지 의존성(Dependency)을 사용자가 직접 하나씩 해결해야 함.

2. DNF (Dandified YUM)

- RPM 기반의 고수준 패키지 관리 도구 (YUM의 후속)
- 핵심 장점: 저장소(Repository) 메타데이터를 활용해 '의존성 자동 해결'
- 저장소 설정 파일 위치: /etc/yum.repos.d/ (*.repo 파일)
- 주요 DNF 명령어:
  - dnf check-update: 업데이트 가능 패키지 확인
  - sudo dnf update: 전체 패키지 업데이트
  - dnf info <pkg>: 패키지 상세 정보 확인
  - dnf list installed / available: 설치된 / 설치 가능 패키지 목록
  - dnf search <keyword>: 키워드로 패키지 이름 및 요약 설명 검색
  - sudo dnf install <pkg> (-y 옵션: 프롬프트 자동 yes 응답)
  - sudo dnf remove <pkg>: 패키지 및 의존 패키지 함께 삭제

(dnf는 독립적인 설치 프로그램이 아니라, 백그라운드에서 rpm을 조종하는 상위 관리자이다)

<4. Apache 웹 서버 설치 및 서비스 구동 실습>

1. Web Server 개념

- HTTP 요청(Request)을 받아 해당 문서를 탐색 후 응답(Response)을 전송하는 서버 SW
- 대표적인 오픈소스 웹 서버: Apache HTTP Server (패키지명: httpd)

2. 웹 서버 구축 순서 (실습 프로세스)

1) 패키지 설치 확인: dnf list installed | grep httpd

2) 패키지 설치: sudo dnf install httpd

3) 서비스 구동 및 자동 실행 설정 (systemctl):

- 서비스 시작: sudo systemctl start httpd
- 상태 확인: sudo systemctl status httpd
- 부팅 시 자동 시작: sudo systemctl enable httpd

4) EC2 보안 그룹 설정: Inbound Rules에서 TCP 80번 포트(HTTP) 개방 필수

5) 웹 브라우저에서 퍼블릭 IP 접속 후 Apache Test Page 확인

(systemctl은 이 서비스들을 켜고, 끄고, 상태를 확인하는 역할을 함 - 서비스관리도구)

CHAPTER 05

5주차 핵심 요약: 리눅스 핵심 명령어 & 시스템 활용

<1. 명령어 기본 구조 & 권한 상승>

1. 명령어 기본 구조

- 형식: command [options] [arguments] (예: ls -l /etc)
- command: 실행할 프로그램 이름
- option: 동작 방식을 바꾸는 옵션 (-로 시작)
- argument: 명령어가 작업할 대상 (파일, 경로 등)
- 매뉴얼 확인: man <command> (종료: q, 검색: /, 이동: b/f/space)
- man이 없는 명령어: <command> --help 로 간단 사용법 확인

2. 권한 상승 (sudo)

- sudo: 관리자(root) 권한이 필요한 작업을 실행할 때 사용
- AWS EC2(ec2-user 등)는 root 비밀번호가 없으며, sudo 그룹에 속해 있어 sudo로만 root 권한 사용
- sudo -i: root 사용자로 전환 (비밀번호 없이 전환 가능)

<2. 표준 입출력 스트림 & 리다이렉션/파이프>

1. 표준 스트림 (Standard Streams)

- #0 stdin (표준 입력): 명령어가 입력을 받는 곳 (기본: 키보드)
- #1 stdout (표준 출력): 명령어가 결과를 출력하는 곳 (기본: 화면)
- #2 stderr (표준 에러): 명령어가 오류 메시지를 출력하는 곳 (기본: 화면)

2. 출력/입력 리다이렉션 (Redirection)

("결과나 에러를 화면에 띄우지 말고, 파일로 방향을 바꿔서 저장해라!")

- `>`: 표준 출력을 파일로 저장 (덮어쓰기)
- `2>`: 표준 에러만 파일로 분리 저장 (예: javac Hello.java 2> error.txt)
- `> output.txt 2> error.txt`: 표준 출력과 에러를 각각 다른 파일로 분리 저장
- `<`: 파일의 내용을 명령어의 입력으로 전달 (예: wc -l < input.txt)

3. 파이프 (Pipe |)

- 앞 명령어의 출력을 뒤 명령어의 입력으로 연결
- 예: ps -ef | grep httpd | wc -l (httpd 관련 프로세스 개수 확인)

<3. 시스템 기본 명령어 & 파일/디렉터리 제어>

1. 시스템 정보 확인 명령어

- pwd: 현재 작업 디렉터리 경로 출력
- whoami: 현재 로그인한 사용자 이름 출력
- who: 현재 시스템에 로그인한 모든 사용자 목록 출력
- which: 명령어의 실행 파일 전체 경로 표시 (예: which whoami -> /usr/bin/whoami)
- su <계정명>: 계정 전환 (su - root, exit로 이전 계정 복귀)

2. 디렉터리 이동 기호 (cd)

- `/` : 최상위 루트 디렉터리
- `.` : 현재 디렉터리
- `..` : 부모(상위) 디렉터리
- `~` : 로그인 사용자의 홈 디렉터리
- `-` : 바로 이전 작업 디렉터리 (cd -)

3. 파일/디렉터리 관리 명령어

- cp from to: 파일 복사
- mv from to: 파일 이동 또는 이름 변경
- touch: 빈 파일 생성 (또는 파일 수정 시각 업데이트)
- rm: 파일 삭제 (-r: 디렉터리 및 내부 하위 요소 재귀적 삭제, -f: 강제 삭제)
- mkdir: 새 디렉터리 생성
- rmdir: '비어 있는' 디렉터리만 삭제

<4. 링크 (Link): 심볼릭 링크 vs 하드 링크 (★시험 단골)>

1. 심볼릭 링크 (Symbolic/Soft Link)

- 생성 명령: ln -s /path/original /path/symlink
- 원본 파일의 '경로(path)'를 가리키는 바로가기 아이콘 역할
- 원본을 삭제하면 경로가 끊어져 링크가 깨짐 (Broken Link)
- 디렉터리 및 다른 파일시스템의 파일에도 생성 가능

> **#차이점 비교 (파일의 구조)**
> - 리눅스 파일 시스템에서 하나의 '파일'은 크게 두 부분으로 나뉘어 저장:
>   원본 파일 (파일명 / 경로):사람이 읽을 수 있는 '이름(예: sample.txt)과 위치'.
>   (실제 데이터가 어디에 저장되어 있는지 가리키는 포인터(이름표) 역할을 함.)
> - 실제 데이터 (inode + 데이터 블록):디스크 상에 실제로 저장된 '글자, 코드, 디스크 공간' 그 자체.
>   (리눅스는 이 실제 데이터를 고유한 inode 번호로 관리.)

2. 하드 링크 (Hard Link)

- 생성 명령: ln /path/original /path/hardlink
- 원본 파일과 동일한 실제 데이터(inode)를 직접 가리킴
- 원본을 삭제해도 하드링크가 남아있으면 데이터가 유지됨
- 디렉터리에는 생성 불가, 같은 파일시스템 내에서만 생성 가능

<5. 파일 내용 확인 명령어>

- cat: 파일 내용을 화면에 그대로 출력
- more: 한 페이지씩 보기 (스페이스바로 다음 페이지)
- less: 위아래 자유 이동 가능 (f/b, j/k, 검색 /, q 종료)
- `head: 파일의 상위 10줄 표시 (head -n N`: N줄 표시)
- tail: 파일의 하위 10줄 표시 (tail -f: 실시간 로그 모니터링에 필수)

<6. 파일 권한 (Permissions) 및 소유권 관리 (★시험 단골)>

1. ls -l 출력 구조 (10자리 권한 문자열)

- 구분 예시: drwxr-xr-x (첫 글자 d: 디렉터리, -: 일반 파일)
- 3글자씩 묶음: User(소유자) / Group(그룹) / Other(기타)

2. 권한 표기 및 8진수 표법

- `Read (r) = 4`: 파일 내용 확인 / 디렉터리 목록 보기(ls)
- `Write (w) = 2`: 파일 내용 수정·삭제 / 디렉터리 내 파일·폴더 생성·삭제
- `Execute (x) = 1`: 파일 실행 / 디렉터리 접근·통과(cd)
- 주의: 디렉터리는 r 권한이 있어도 x 권한이 없으면 디렉터리 안으로 진입(cd) 불가!
- 8진수 계산 예시: rw-rw-r-- = (4+2+0)(4+2+0)(4+0+0) = 664

3. 권한 변경 (chmod)

- 기호 모드: chmod a+x filename (모두에게 실행 권한 추가), chmod u-rx filename
- 숫자 모드: chmod 664 filename

4. 소유자/그룹 변경 (chown / chgrp)

- `sudo chown ec2-user sample.txt`: 소유자 변경
- `sudo chown ec2-user:webteam sample.txt`: 소유자와 그룹 동시 변경
- `sudo chgrp webteam sample.txt`: 그룹만 변경

<7. 고급 파일/텍스트 검색 및 가공 도구>

1. find (파일 검색)

- `find -name "*.java"`: 파일명 조건 검색
- `find /var/log -type f -mtime -1`: 최근 24시간 이내 수정된 일반 파일 검색 (-mtime -1: 0~24시간, 1: 24~48시간, +1: 48시간 이상)
- `find -size +100M`: 100MB 초과 파일 검색

2. grep (텍스트 패턴 검색)

- grep "패턴" 파일명
- 주요 옵션:
  - `-r` : 하위 디렉터리 재귀적 검색
  - `-n` : 매칭된 줄 번호 함께 표시
  - `-i` : 대소문자 구분 없이 검색
- 파이프라인 조합: find . -name "*.java" | xargs grep -i "hello"

(xargs를 통해 find 결과 파일들을 grep의 인수로 전달)

(xargs는 앞 명령어의 출력 결과(텍스트)를 받아서, 뒤 명령어의 '인자'로 나누어 전달해 주는 변환 도구)

(즉, 각 .java 파일의 내부 내용을 열어서 "hello"를 검색!!)

3. sed & awk (텍스트 가공)

- sed (Stream Editor): 텍스트 치환/가공
  - sed 's/foo/bar/g' file.txt (화면에 출력만 함, 원본 유지)
  - sed -i 's/foo/bar/g' file.txt (-i: 원본 파일 자체를 덮어씀. 백업용: sed -i.bak)
- awk (필드 기반 처리): 공백/구분자 기준 열(column) 단위 추출
  - awk '{print $1}' access.log (첫 번째 필드인 IP 주소만 출력)
  - $1: 1번째 필드, $NF: 마지막 필드

<8. 압축 관리 (tar) & 셸 스크립트>

1. tar (테이프 아카이브 & 압축)

- 압축 생성: tar -czvf archive.tar.gz /path/directory
  - -c: 아카이브 생성, -z: gzip 압축, -v: 진행 과정 출력, -f: 파일명 지정
- 압축 해제: tar -xzvf archive.tar.gz (-x: 압축 해제)

2. 셸 스크립트 (Shell Script)

- 스크립트 첫 줄: #!/bin/bash (shebang: 이 스크립트를 실행할 셸 지정)
- 실행 방법: chmod a+x script.sh (실행 권한 부여) 후 ./script.sh 실행

<9. 웹 통신 도구 (wget vs curl) & 환경 설정>

1. wget vs curl 차이점 (★시험 단골)

- wget: 웹/FTP 파일 다운로드 전용 도구 (기본 동작이 파일 저장)
- curl: 다양한 프로토콜(HTTP, FTP, SMTP 등) 통신 범용 클라이언트.
  - 파일 저장은 -O 옵션으로 실행 (curl -O url)
  - 응답 헤더만 확인(curl -I url) 또는 데이터 POST 전송(curl -X POST ...) 등 다운로드 외 다양한 네트워크 요청 가능
- AL2023 등 최신 리눅스에는 curl이 기본 설치, wget은 별도 설치 필요할 수 있음

2. 로그인 사용자 환경 설정 파일 (.bash_profile vs .bashrc)

- .bash_profile: SSH 로그인 시 '최초 1회' 실행 (로그인 세션 초기화)
- .bashrc: 새 터미널/서브셸을 열 때마다 반복 실행 (alias, PATH 추가 등 개인화 설정)

- 로그인 시에는 .bash_profile 실행 과정 내부에서 .bashrc를 불러오므로 둘 다 실행됨
- alias 지정: alias ll='ls -alf'
- PATH 환경변수 설정: export PATH="/opt/myapp/bin:$PATH" (기존 PATH 앞에 추가)
- env: 전체 환경변수 목록 확인

CHAPTER 06

[Week 6] Git & GitHub - 기초 요약 노트

<1. VCS (버전 관리 시스템) 개념>

- 백업, 이력 관리, 협업을 가능하게 하는 프로젝트 관리 도구
- 분산형 VCS (Git):
  - 모든 개발자가 프로젝트 전체 사본(로컬 저장소)을 보유
  - 서버가 다운되거나 오프라인 상태여도 로컬에서 커밋/조회 가능
- Git 저장 방식 (Snapshot):
  - 파일의 변경분(Delta)만 저장하는 기존 SVN 방식과 달리, 매 커밋마다 프로젝트 전체의 '스냅샷'을 저장함

<2. Git 초기 설정 (git config)>

```
# 사용자 정보 등록 (최초 1회)
git config --global user.name "이름"
git config --global user.email "이메일"

# 기본 브랜치명을 main으로 지정 (최신 관례)
git config --global init.defaultBranch main

# 전체 설정 확인
git config --list
```

<3. Git 3단계 영역 & 워크플로우 (Workflow)>

1) Working Directory (작업 디렉터리): 코드를 수정하는 실제 공간

2) Staging Area (스테이징 영역): 커밋에 담을 변경 사항을 골라 담는 임시 공간

3) Local Repository (로컬 저장소): 커밋 이력이 영구 저장되는 공간

- 기본 순환 구조:
  [코드 수정] -> [git add] (스테이징) -> [git commit] (로컬 저장소 저장)

<4. 필수 명령어 요약>

`git init`

현재 디렉터리를 Git 저장소로 초기화 (.git 숨김 폴더 생성)

`git status`

파일들의 현재 상태 (Untracked, Modified, Staged 등) 확인

`git add <file>`

변경된 파일/디렉터리를 스테이징 영역에 추가 (git add . 은 전체 추가)

`git commit -m "커밋 메시지"`

스테이징 영역의 파일들을 로컬 저장소에 커밋

`git log`

커밋 히스토리 조회 (git log --oneline 으로 한 줄 요약)

`git diff`

Working Directory vs Staging Area 변경 내용 비교

`git diff --cached`

Staging Area vs HEAD(최신 커밋) 변경 내용 비교

`git diff HEAD`

Working Directory vs HEAD 전체 변경 내용 비교

<5. .gitignore (특정 파일 무시하기)>

- 빌드 산출물(*.class, *.exe), 설정 파일 등 추적 불필요한 파일 제외
- 프로젝트 최상단 디렉터리에 .gitignore 파일 생성 후 패턴 적기

  예시:

```
*.class      # 확장자가 .class인 파일 제외
temp/        # temp 디렉터리 전체 제외
```

CHAPTER 07

[Week 7] Git & GitHub - 원격 저장소 전체 요약 노트

<1. 원격 저장소(Remote Repository) 개념>

- 로컬 저장소: 내 컴퓨터(.git)에 존재하는 커밋 이력
- 원격 저장소: GitHub, GitLab 등 인터넷 서버에 호스팅된 저장소
- 주요 역할: 코드 백업 및 다른 개발자와의 협업을 위한 중심 축

<2. GitHub 계정 및 원격 저장소 생성>

- 계정 생성 및 보안: 2단계 인증(2FA) 설정 권장 (보안 의무화 추세)
- New Repository 생성 시 주의사항:
  - 이미 로컬에 프로젝트가 있는 경우: README, .gitignore, License를 추가하지 않고 '빈 저장소'로 생성 (충돌 방지)
  - 기본 브랜치명: main (GitHub 최신 관례)

<3. 로컬 저장소와 원격 저장소 연결 (git remote)>

```
# 원격 저장소 주소 등록 (origin은 관례적인 기본 별칭)
git remote add origin https://github.com/사용자명/저장소명.git

# 연결된 원격 저장소 목록 및 URL 확인
git remote -v

# 원격 저장소 이름 변경 및 연결 해제
git remote rename origin upstream
git remote remove upstream
```

<4. GitHub 인증 - Personal Access Token (PAT)>

- 2021년 8월부터 계정 비밀번호를 통한 git push/pull 사용 불가 (PAT 필수)
- PAT 특징: 비밀번호 대용 긴 문자열(ghp_...), 권한(scope) 제한 및 만료일 설정 가능
- 발급 경로: Settings -> Developer settings -> Personal access tokens -> Tokens (classic)
- 권한 설정: public_repo (공개 저장소) 또는 repo (비공개 포함 전체 권한)
- 인증 사용법: git push/pull 시 비밀번호 입력 창에 발급받은 PAT 문자열 붙여넣기

<5. push & pull & clone 핵심 명령어>

```
# 로컬 커밋을 원격 저장소로 업로드 (-u 옵션은 최초 1회만 사용, upstream 연결)
git push -u origin main

# -u 설정 이후 사용 가능한 간단한 명령어
git push
git pull

# 원격 저장소 전체를 내 컴퓨터로 복제 (origin 별칭 자동 연결)
git clone https://github.com/사용자명/저장소명.git
```

<6. 태그(Tag)로 릴리스 지점 표시하기>

- 태그 개념: 특정 커밋에 붙이는 고정된 이름표 (주로 v1.0.0 등 릴리스 버전에 사용)
- 주석 태그(Annotated Tag) 생성 (-a: 작성자, 날짜, 메시지 포함)

```
git tag -a v1.0.0 -m "First stable release"

# 특정 과거 커밋(해시값)에 태그 달기
git tag -a v0.9.0 a1b2c3d -m "Beta release"

# 태그 목록 조회 및 상세 정보 확인
git tag
git show v1.0.0

# 태그를 원격 저장소에 업로드 (git push만으로는 태그가 올라가지 않음)
git push origin v1.0.0      # 특정 태그 하나만 push
git push origin --tags       # 모든 태그를 한 번에 push
```

<7. 2인 협업 워크플로우 요약>

1) Owner: GitHub 저장소 생성 -> Collaborators에 상대방 추가

2) Invited User: 초대 수락 -> git clone으로 저장소 복제

3) 파일 수정 -> git add -> git commit -> git push (PAT 인증)

4) Owner: git pull로 상대방의 변경 사항 로컬로 다운로드 및 반영